Module 19 — Learning Roadmap & Interview Study Plan

> **Goal:** Turn the repository into a schedule you can actually execute: know which domains your target role weights, what Mid, Senior and Staff answers sound like, how to study with retrieval practice rather than re-reading, how to score your own mock interviews, which primary sources to read, how to structure behavioural stories, and a compact glossary and numbers sheet for last-minute review.

Table of Contents

- 19.1 Skill Map and Level Rubrics: Mid, Senior, Staff
- 19.2 Study Plans: 72 Hours, 2 Weeks, 6 Weeks and 12 Weeks
- 19.3 Self-Assessment Rubrics and Practice Scoring
- 19.4 Foundational Reading List: Papers, Books and Documents
- 19.5 Loop Anatomy and Behavioural Stories
- 19.6 Glossary and Numbers Sheet

19.1 Skill Map and Level Rubrics: Mid, Senior, Staff

Conceptual Deep Dive

LLMOps interviews sample twelve competency domains. Pointers are to modules: (1) memory and cost maths [01, 12]; (2) data and synthetic data [02]; (3) prompting and structured output [03]; (4) compression and PEFT [04, 15]; (5) serving [05]; (6) retrieval [06]; (7) evaluation [07, 14]; (8) safety and security [08, 16]; (9) agents [09]; (10) observability and incident response [10, 17]; (11) cost, capacity and platform [11, 17]; (12) release engineering and governance [13, 16].

The levels differ in kind, not just breadth. **Mid** explains concepts correctly and implements with guidance. **Senior** chooses between options using measured trade-offs, anticipates failure modes and owns a component end to end. **Staff** frames ambiguous problems, sets standards across teams, quantifies business impact, designs for evolution under organisational constraints, and mentors. Staff is a change in scope and influence, not "Senior with more tools".

Signal	Mid	Senior	Staff
Scope	A task	A component	A system or several teams
Ambiguity	Asks for the spec	Proposes a spec	Decides what not to build
Depth on demand	Definitions	Mechanisms and numbers	Mechanisms plus second-order effects
Quantification	Rough	Worked arithmetic	Arithmetic tied to cost and risk

Worked contrast, serving: asked why TTFT is high, a Mid answer says "the model is big, add GPUs". A Senior answer separates queueing from prefill, checks queue depth and prefix-cache hit rate, and proposes chunked prefill (Module 05 section 1.5 (legacy)). A Staff answer adds the policy layer: SLO tiers with admission control, capacity headroom, and a benchmark standard (5.8) so teams compare engines honestly. Evaluation: Mid computes a pass rate; Senior adds a Wilson interval and judge calibration (Module 07 sections 2.5, 2.7 (legacy)); Staff defines the gate policy, ownership of the golden set and how offline metrics are validated

against online outcomes (14.6).

Role profiles weight domains differently: application engineers lean on prompting, retrieval, evaluation and agents; ML platform engineers on serving, cost, platform and release; SRE or infrastructure on serving, observability and capacity; security-minded engineers on 8, 9 and governance; engineering managers on evaluation, cost, release and team topology (1.6). Use the map by self-scoring 0-4 per domain with an evidence note, taking the two weakest high-weight domains, then schedule them in 19.2 and drill them in 19.3.

Not expected at any level: deriving CUDA kernels, reproducing paper proofs or memorising vendor prices. Edge cases: a strong specialist presents as T-shaped, naming one deep domain with a worked example and being honest about others. MLOps or SRE candidates map existing strengths (rollouts, SLOs, on-call) onto domains 10-12 and spend study time on 5-7.

Cross-Connection Scenarios

- Rubric dimensions here feed the practice scoring in 19.3 and the mock format in 20.1.
- Domain 7 deepens through the eval pyramid (14.1) and error analysis (7.7).
- Domain 11 connects to FinOps (11.8) and the multi-tenant design (20.3).
- The estimation drills in 12.6 are the quickest way to show Senior-level quantification.

Production Anti-Patterns

- Studying every domain equally regardless of target role: wasted hours on low-weight domains while the weighted gaps stay open.
- Treating Staff as "Senior with more tools": answers stay at component scope and miss the standards, gates and org mechanisms that Staff interviews probe.
- Self-scoring without evidence: inflated scores hide the real gaps until a live interview exposes them.

Minimal Working Code

A skill matrix with per-role weights, evidence-capped self-scores and a weighted-gap prioritiser.

```
LEVEL_TARGET = {"mid": 2, "senior": 3, "staff
": 4}
DOMAINS = ["memory", "data", "prompting", "compression", "se
rving", "retrieval",
           "evaluation", "security", "agents", "observability", &
quot;cost_platform", "release_gov"]
ROLES = {  # weight 0-3 per domain, same order as DOMAINS
    "application": [2, 1, 3, 1, 1, 3, 3, 2, 3, 2, 1, 2],
    "platform":    [3, 1, 1, 2, 3, 1, 2, 2, 1, 3, 3, 3],
    "sre":         [2, 0, 0, 1, 3, 0, 1, 2, 0, 3, 3, 3],
    "security":    [1, 1, 2, 0, 1, 2, 2, 3, 3, 2, 1, 3],
}

def effective(score, evidence):
    """A score above 2 without an evidence note is capped at 2."""
    return min(score, 2) if score > 2 and not evidence.strip() else score

def priorities(role, level, self_scores, top=3):
    w = dict(zip(DOMAINS, ROLES[role]))
    gaps = {d: w[d] * max(0, LEVEL_TARGET[level] - effective(s, e))
            for d, (s, e) in self_scores.items()}
    return sorted(gaps.items(), key=lambda kv: (-kv[1], kv[0]))[:top]

me = {d: (3, "owned it in prod") for d in DOMAINS}
me["serving"] = (4, "")                 # claimed, no evidence: capped to 2
me["evaluation"] = (1, "read module 07")
top = priorities("application", "senior", me)
print(top)
```

```
assert top[0] == ("evaluation", 6) and top[1] == ("serving", 1)
```

Turn gaps into a reading queue ordered by weight times gap.

```
READ = {
    "serving": ["Module 05 sections 1.3-1.6 (legacy)", "5.6", "5.8&quo
t;],
    "evaluation": ["Module 07 sections 1.5, 2.5-2.7 (legacy)", "7.6", "
;7.7", "14.1"],
    "retrieval": ["Module 06 sections 1.1, 1.5, 1.6 (legacy)", "6.8"],
    "security": ["Module 08 sections 1.2, 1.3 (legacy)", "8.6", "8.8&q
uot;],
    "agents": ["Module 09 sections 1.1, 2.1 (legacy)", "9.7"],
}

def queue(weights, gaps):
    rows = sorted(((weights[d] * g, d) for d, g in gaps.items() if g > 0), reverse=True)
    return [(score, d, READ.get(d, ["(no pointer)"])) for score, d in rows]

weights = {"serving": 3, "evaluation": 3, "retrieval": 3, "security&qu
ot;: 2, "agents": 1}
gaps = {"serving": 1, "evaluation": 2, "retrieval": 0, "security"
: 2, "agents": 1}
for score, domain, sections in queue(weights, gaps):
    print(score, domain, sections)
assert queue(weights, gaps)[0][1] == "evaluation"
```

Interview Angle

Q: What distinguishes a Staff answer from a Senior answer on "how would you improve our RAG quality?" **A:** Senior: slice failures by retriever versus generator (7.7), fix the largest slice, and report recall@k and faithfulness with intervals. Staff does that, then asks what decision the metric serves, sets up the golden-set ownership, gate and flywheel (14.1, 14.4) so improvement outlasts one engineer, and prices the change per resolved query. Check: a before/after on a frozen set, plus an online metric that moves.

Q: How do you demonstrate breadth and depth within a 45-minute loop? **A:** Give a structured skeleton fast (CRISP-LM, Module 12 section 1.2 (legacy)), then pick one or two branches to take to mechanism level with numbers, signalling the choice aloud. Trade-off: breadth earns the framing marks, depth earns the level; spending more than ten minutes on one branch costs coverage. Check: in a recorded mock, every rubric dimension is touched and at least one has arithmetic.

Q: Which domains for a platform-engineering role? **A:** Serving, cost and capacity, observability, release and multi-tenancy first (weights 3), retrieval and prompting lighter. The trade-off is depth in application quality versus operability; I would still keep evaluation at 2 because platform decisions need quality gates. Check: weighted gap score from the matrix, revisited after each mock.

19.2 Study Plans: 72 Hours, 2 Weeks, 6 Weeks and 12 Weeks

Conceptual Deep Dive

Every plan is a table of **day, read, do, say aloud, verify, time**. Aim for roughly 40% reading, 30% building and 30% speaking or mocking. The evidence for study technique is consistent: **retrieval practice and spaced repetition** beat re-reading, and explain-aloud fluency and whiteboard estimation are skills that decay without rehearsal. Review weekly, compare against your 19.1 gaps, and move hours from comfortable topics to weak ones.

72-hour plan (roughly 6 hours per day plus sleep). The cut list is explicit: skip training internals, kernels, MoE detail, graph retrieval and governance detail beyond one paragraph.

Block	Read	Do	Say aloud	Verify
Day 1	Numbers sheet: Module 01 sections 2.2-2.3, Module 12 section 1.6 (legacy), 19.6; serving: Module 05 sections 1.3-1.6	Re-derive KV bytes per token and one GPU count	Why decode is bandwidth-bound	Compute 70B KV per token cold
Day 2	RAG and eval: Module 06 sections 1.1, 1.5, 1.6; Module 07 sections 1.2, 1.5, 2.5; safety: Module 08 sections 1.2-1.3	Sketch hybrid retrieval with ACL filters	The eval triad and judge biases	Name three prompt-injection controls
Day 3	CRISP-LM (Module 12 section 1.2 (legacy)); 20.2; stories 19.5	One timed 45-minute mock on 20.2	Two stories in four minutes each	Score the recording with 19.3

2-week plan: add quantisation and PEFT (Module 04 sections 1.1-1.5), agents (Module 09 sections 1.1-1.8), observability and SLOs (Module 10 sections 1.6, 2.3) and cost (Module 11 sections 1.3-1.4); two mocks. **6-week plan:** all legacy modules with labs 18.1-18.3, a weekly mock and spaced review of weak cards. **12-week plan:** add modules 13-17, build projects 18.4-18.6, publish a write-up and run mock loops with peers. Part-time (6-8 hours per week): halve each phase, keep one mock per fortnight, and never drop the review slots.

Interview moved up? Triage in this order: numbers, serving, RAG and eval, one design, stories; then safety; then everything else as flashcards only. Sleep is part of the plan: cramming overnight lowers recall and delivery more than an extra hour of reading gains.

Cross-Connection Scenarios

- Reading targets come from the skill map (19.1) and the question bank (20.7).
- Each "verify" step can be scored with the rubric in 19.3.
- Lab time uses the projects in 18.1-18.6; log trade-offs as you go.
- Flashcards should be built from the glossary (19.6) and the must-say lists in 20.7.

Production Anti-Patterns

- Passive reading only: familiarity feels like knowledge, then recall fails under interview pressure.
- Building a project without writing down trade-offs: you cannot explain why, so the project adds no interview evidence.
- Cramming the night before with no sleep: delivery and arithmetic accuracy degrade exactly when they are scored.

Minimal Working Code

A day-by-day schedule generator that allocates days by weight and adds spaced review slots.

```
import datetime as dt

def plan(days, weights, start, review_offsets=(1, 3, 7)):
    total = sum(weights.values())
    raw = {d: days * w / total for d, w in weights.items()}
    alloc = {d: int(v) for d, v in raw.items()}
    leftover = days - sum(alloc.values())
    for d in sorted(raw, key=lambda k: raw[k] - alloc[k], reverse=True)[:leftover]:
        alloc[d] += 1
    order = [d for d in sorted(alloc, key=lambda k: -weights[k]) for _ in range(alloc[d])]
    sched = {i: [("learn", t)] for i, t in enumerate(order)}
    for i, t in enumerate(order):
        for off in review_offsets:
            if i + off < days:
                sched[i + off].append(("review", t))
    return [(start + dt.timedelta(days=i), items) for i, items in sched.items()]
```

```
rows = plan(6, {"serving": 3, "rag": 2, "eval": 1}, dt.date(2026, 11, 2))
for day, items in rows:
    print(day, items)
assert len(rows) == 6 and ("review", "serving") in rows[1][1]
```

A Leitner-box scheduler over JSON flashcards, with promotion on success and demotion to box 1 on failure.

```
import datetime as dt
import json

BOX_DAYS = {1: 1, 2: 2, 3: 4, 4: 8, 5: 16}
cards = json.loads('[{"id": "kv", "box": 1, "due": "2026-11-
02"},'
                   ' {"id": "rrf", "box": 3, "due": "2026-11
-05"}]')

def due(cards, today):
    return [c["id"] for c in cards if dt.date.fromisoformat(c["due"]) <= today]

def review(cards, today, results):
    for c in cards:
        if c["id"] in results:
            c["box"] = min(5, c["box"] + 1) if results[c["id"]] else 1
            c["due"] = (today + dt.timedelta(days=BOX_DAYS[c["box"]])).isoformat()

today = dt.date(2026, 11, 2)
assert due(cards, today) == ["kv"]
review(cards, today, {"kv": True})
assert cards[0]["box"] == 2 and cards[0]["due"] == "2026-11-04"
review(cards, dt.date(2026, 11, 5), {"rrf": False})
assert cards[1]["box"] == 1 and "rrf" in due(cards, dt.date(2026, 11, 6))
print(cards)
```

Interview Angle

Q: You have 5 days before a Staff LLMOps loop. What do you study and drop? **A:** Days 1-2: numbers, serving, retrieval and evaluation to mechanism level; day 3: safety, release gates and one cost lever set; day 4: two timed designs (20.2, 20.3); day 5: stories, light review, sleep. Drop training internals and breadth-only reading. Trade-off: depth on the likely design prompts versus coverage. Check: a recorded mock scoring at least 3 on quantification and failure thinking (19.3).

Q: How do you know you are ready for a system-design round? **A:** Three consecutive timed mocks where the sizing is correct without notes, every rubric dimension is touched, and a peer cannot find an unaddressed failure mode. Trade-off: more mocks versus diminishing returns; stop when scores plateau and fix the lowest dimension instead.

Q: How do you convert reading into spoken fluency? **A:** Close the page, explain the section in three minutes into a recording, compare to the must-say list (20.7), and repeat on the spaced schedule. Check: keyword coverage of at least 80% and no filler stalls on arithmetic.

19.3 Self-Assessment Rubrics and Practice Scoring

Conceptual Deep Dive

Score each mock on nine design dimensions, 1-4, with one-sentence **anchor** descriptions. Anchors stop you scoring on vibes.

Dimension	1	2	3	4
Problem framing	Starts with tools	Asks a few questions	Clarifies users, scale, SLOs	Also states out-of-scope and success metrics
Quantification	No numbers	Numbers without arithmetic	Worked arithmetic with units	Sensitivity and cost per outcome

Dimension	1	2	3	4
Architecture	Boxes without flows	Plausible flow	Data and control planes separated	Evolution path and ownership
Trade-offs	One option	Names alternatives	Chooses with criteria	States what would flip the decision
Failure thinking	None	Lists failures	Detection and mitigation	Blast radius and degradation ladder
Evaluation	None	"We will test"	Golden set, metrics, gates	Online-offline validation
Security	None	Generic guardrails	Trust boundaries, injection, ACLs	Threat-model driven controls
Operations and cost	Ignored	Mentions monitoring	SLOs, rollout, unit cost	Capacity plan and runbooks
Communication	Rambling	Clear in parts	Structured, signposted	Adapts to pushback concisely

Scoring a recorded mock: pause at each phase, score each dimension with an evidence timestamp, and compare with a peer's independent scores. Compute **inter-rater agreement** (exact-match rate, or Cohen's kappa for calibration) on the nine scores; if the two raters disagree by two points on a dimension, re-read the anchor together. Map the weighted mean to a level only as an indicative threshold and cap the level if quantification or failure thinking is below 3, since those are the dimensions that distinguish Senior from Mid.

Drill types: 5-minute concept explanation, 10-minute estimation (12.6), 45-minute design, 20-minute debugging (20.8) and a behavioural story (19.5), each with a trimmed rubric. Track trends per dimension and aim deliberate practice at the lowest one rather than favourite topics. Common failure patterns and fixes: tool-first answers (start with requirements), no numbers (always finish a sizing block), no failure modes (add a failure pass at minute 25), no rollout (close with canary and rollback).

Peer-mock protocol: roles of candidate, interviewer and observer; a timekeeper; feedback as observation, impact, suggestion ("you gave no QPS; I could not judge the GPU count; state it before choosing hardware"). Edge cases: self-bias, since candidates over-rate communication, so use recordings; peers of different seniority, so calibrate to the target level and weight the more experienced reviewer's evidence, not just the score.

Cross-Connection Scenarios

- The 45-minute format and probing ladder are in 20.1.
- Dimension 2 is trained with the estimation drills in 12.6.
- Dimensions 5 and 6 map to the symptom drills in 20.8 and eval operations in 14.1.
- Behavioural scoring uses the story structure in 19.5.

Production Anti-Patterns

- Scoring on vibes: unanchored scores drift upward and hide the same weakness for weeks.
- Always practising favourite topics: weak dimensions never improve and surface in the real loop.
- No recording: you cannot collect timestamp evidence or notice filler, monologues and skipped numbers.

Minimal Working Code

A rubric scorer with weights, level thresholds with caps and a weakest-dimension report.

```
DIMS = {"framing": 1, "quantification": 2, "a
rchitecture": 1, "tradeoffs": 2,
        "failure": 2, "evaluation": 1, "security": 1, "ops_cost":
 1, "communication": 1}
LEVELS = [(3.3, "staff-track"), (2.7, "senior"), (2.2, "mid"), (0, "be
low mid")]

def score(scores):
    mean = sum(DIMS[d] * s for d, s in scores.items()) / sum(DIMS.values())
    level = next(name for cut, name in LEVELS if mean >= cut)
```

```
    if level in ("staff-track", "senior") and min(scores["quantification"],
 scores["failure"]) < 3:
        level = "mid"          # indicative cap: these two separate Senior from Mid
    weakest = sorted(scores, key=lambda d: (scores[d], -DIMS[d]))[:2]
    return round(mean, 2), level, weakest

s = {d: 3 for d in DIMS}
s["quantification"] = 2
print(score(s))
assert score(s)[1] == "mid" and score(s)[2][0] == "quantification"
assert score({d: 4 for d in DIMS})[1] == "staff-track"
```

A mock-session logger that appends CSV rows and prints a per-dimension trend with the next drill.

```
import csv
import io

DRILL = {"quantification": "10-minute estimation drill", "failure": "2
0-minute debugging drill"}

def summarise(text):
    rows = list(csv.DictReader(io.StringIO(text)))
    by_dim = {}
    for r in rows:
        by_dim.setdefault(r["dimension"], []).append(int(r["score"]))
    trend = {d: (v[-1] - v[0], v[-1]) for d, v in by_dim.items()}
    lowest = min(trend, key=lambda d: (trend[d][1], trend[d][0]))
    return trend, DRILL.get(lowest, "full 45-minute design on " + lowest)

buf = io.StringIO()
w = csv.writer(buf)
w.writerow(["date", "dimension", "score"])
for row in [("d1", "quantification", 2), ("d1", "failure", 3),
            ("d2", "quantification", 2), ("d2", "failure", 3)]:
    w.writerow(row)
trend, nxt = summarise(buf.getvalue())
print(trend, nxt)
assert nxt == "10-minute estimation drill"
```

Interview Angle

Q: How do you give a peer useful feedback after a mock design interview? **A:** Use observation, impact, suggestion tied to a rubric dimension and a timestamp: "At minute 22 you picked a vector index without a memory estimate; I could not tell if it fits; size it first." Trade-off: specific criticism versus morale, so lead with one strength. Check: the peer's score on that dimension rises in the next mock.

Q: You score 2/4 on quantification repeatedly. How do you fix it? **A:** Run daily five-minute estimation drills from 12.6 with a fixed template (users, peak QPS, tokens, memory, GPUs, cost), then state assumptions in a table. Trade-off: speed versus precision; whiteboard accuracy of about 20% is enough. Check: sizing completed in under seven minutes with units.

Q: How do you calibrate yourself against real interviewers? **A:** Have a senior peer score the same recording blind, compute agreement, and adjust anchors where you diverge by two points. Compare with feedback from real loops. Check: agreement of at least 70% exact-or-adjacent across nine dimensions.

19.4 Foundational Reading List: Papers, Books and Documents

Conceptual Deep Dive

Read fewer works, more deliberately. Use three passes: abstract and figures, method, limitations. Then write one paragraph, an "interview hook", linking the work to a decision you would defend. Papers capture a moment in time, so pair them with current documentation for any tool. This list gives title, authors and year only; it is not a substitute for reading the sources, and venues are deliberately omitted because many items circulated as pre-prints before publication. For standards and regulation, verify the current

text.

Foundations

Work	Authors, year	Why read	Repo sections
Attention Is All You Need	Vaswani et al., 2017	The architecture behind every serving calculation	Module 01 sections 2.1-2.3 (legacy)
Language Models are Few-Shot Learners	Brown et al., 2020	In-context learning as a prompting foundation	Module 03
Scaling Laws for Neural Language Models	Kaplan et al., 2020	Why compute, data and size trade off	Module 01
Training Compute-Optimal Large Language Models	Hoffmann et al., 2022	Data versus parameter budgeting	Module 02, 15.1
Training language models to follow instructions with human feedback	Ouyang et al., 2022	The RLHF pipeline	15.3
Mixtral of Experts	Jiang et al., 2024	Sparse expert models and their serving cost	5.7

Serving

Work	Authors, year	Why read	Repo sections
Efficient Memory Management for Large Language Model Serving with PagedAttention	Kwon et al., 2023	Block-based KV memory management	Module 05 section 1.3 (legacy)
Orca: A Distributed Serving System for Transformer-Based Generative Models	Yu et al., 2022	Iteration-level scheduling	Module 05 section 1.4 (legacy)
FlashAttention	Dao et al., 2022	IO-aware attention kernels	Module 01 section 2.7 (legacy)
Fast Inference from Transformers via Speculative Decoding	Leviathan, Kalman, Matias, 2022	Draft-and-verify decoding	Module 05 section 2.5 (legacy)
Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve	Agrawal et al., 2024	Chunked prefill scheduling	Module 05 section 2.6 (legacy)
DistServe	Zhong et al., 2024	Prefill and decode disaggregation	Module 05 section 1.9 (legacy)
SGLang	Zheng et al., 2023	Prefix reuse and structured programs	5.6

Compression, PEFT, alignment, data

Work	Authors, year	Why read	Repo sections
LoRA	Hu et al., 2021	Low-rank adapters	Module 04 section 2.6 (legacy)
QLoRA	Dettmers et al., 2023	4-bit base plus adapters	Module 04 section 3.6 (legacy)
GPTQ	Frantar et al., 2022	Second-order post-training quantisation	Module 04 section 2.3 (legacy)
AWQ	Lin et al., 2023	Activation-aware scaling	Module 04 section 2.4 (legacy)

Work	Authors, year	Why read	Repo sections
SmoothQuant	Xiao et al., 2022	Moving activation outliers into weights	4.6
Direct Preference Optimization	Rafailov et al., 2023	Preference learning without a reward model	15.3
Constitutional AI	Bai et al., 2022	Principle-driven feedback	Module 08
DeepSeekMath	Shao et al., 2024	Group-relative RL for reasoning	15.4
Self-Instruct	Wang et al., 2022	Bootstrapped instruction data	Module 02 section 3.7 (legacy)

Retrieval and RAG

Work	Authors, year	Why read	Repo sections
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Lewis et al., 2020	The original RAG framing	Module 06
Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs	Malkov and Yashunin, 2016	The HNSW index	Module 06 section 1.3 (legacy)
Lost in the Middle	Liu et al., 2023	Position effects in long contexts	3.6
Reciprocal Rank Fusion	Cormack, Clarke, Buttcher, 2009	Rank fusion for hybrid search	Module 06 section 2.4 (legacy)
From Local to Global: A Graph RAG Approach to Query-Focused Summarization	Edge et al., 2024	Graph-based retrieval	6.6

Evaluation, agents, security

Work	Authors, year	Why read	Repo sections
Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena	Zheng et al., 2023	Judge biases and agreement	Module 07 section 1.5 (legacy)
RAGAS	Es et al., 2023	Reference-free RAG metrics	Module 07 section 3.3 (legacy)
Holistic Evaluation of Language Models	Liang et al., 2022	Multi-metric evaluation design	14.1
ReAct	Yao et al., 2022	Reason-and-act loops	Module 09
Chain-of-Thought Prompting Elicits Reasoning in Large Language Models	Wei et al., 2022	Reasoning in prompts	Module 03 section 1.5 (legacy)
Toolformer	Schick et al., 2023	Learned tool use	9.7
Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection	Greshake et al., 2023	Indirect injection	Module 08 section 1.2 (legacy)

Work	Authors, year	Why read	Repo sections
Universal and Transferable Adversarial Attacks on Aligned Language Models	Zou et al., 2023	Automated adversarial suffixes	8.8
Extracting Training Data from Large Language Models	Carlini et al., 2021	Memorisation and leakage	Module 08 section 2.6 (legacy)
Defeating Prompt Injections by Design	Debenedetti et al., 2025	Architectural injection defence	9.6

Systems and governance

Work	Authors, year	Why read	Repo sections
Hidden Technical Debt in Machine Learning Systems	Sculley et al., 2015	ML system maintenance costs	1.6
The Tail at Scale	Dean and Barroso, 2013	Tail latency	Module 10 section 2.2 (legacy)
Site Reliability Engineering	Beyer, Jones, Petoff, Murphy, 2016	SLOs and incident practice	Module 10 section 2.3 (legacy), 17.6
Designing Data-Intensive Applications	Kleppmann, 2017	Data systems foundations	Module 06, 9.8
Model Cards for Model Reporting	Mitchell et al., 2019	Model documentation	16.3
Datasheets for Datasets	Gebru et al., 2018	Dataset documentation	16.3
NIST AI Risk Management Framework 1.0	NIST, 2023	Risk framework	16.2
Regulation (EU) 2024/1689 (EU AI Act)	European Union, 2024	Binding regulation; verify current text	16.2
ISO/IEC 42001:2023	ISO/IEC, 2023	AI management system standard	16.1

Also read engineering write-ups and documentation from the serving engines and vector stores you will name in an interview.

Cross-Connection Scenarios

- The serving papers explain the numbers in Module 05 sections 2.1-2.6 (legacy).
- Retrieval entries pair with 6.6-6.8 and the funnel in Module 06 section 1.1 (legacy).
- Safety entries pair with the OWASP mapping in 8.6 and the red-team pipeline in 8.8.
- Governance entries feed 16.1-16.3.

Production Anti-Patterns

- Reading 40 papers shallowly: no interview hook forms, so none survives a probing question.
- Quoting results from memory in interviews: misquoted numbers destroy credibility; state mechanisms and say you would check the paper.
- Ignoring engineering write-ups and docs: you describe a system as it was in the paper, not as it behaves in the current release.

Minimal Working Code

A reading tracker with a time-boxed plan generator.

```
from dataclasses import dataclass

@dataclass
class Entry:
    title: str
    minutes: int
```

```
    status: str = "todo"
    hook: str = ""

def time_boxed(entries, daily_minutes):
    days, today, used = [], [], 0
    for e in (x for x in entries if x.status == "todo"):
        if used + e.minutes > daily_minutes and today:
            days.append(today)
            today, used = [], 0
        today.append(e.title)
        used += e.minutes
    return days + ([today] if today else [])

items = [Entry("PagedAttention", 90), Entry("Orca", 60), Entry("LoRA", 60,
"done"),
         Entry("Lost in the Middle", 45)]
plan = time_boxed(items, 120)
print(plan)
assert plan == [["PagedAttention"], ["Orca", "Lost in the Middle"]]
```

A citation-entry validator that flags missing fields and duplicates.

```
def validate(entries):
    problems, seen = [], set()
    for i, e in enumerate(entries):
        for field in ("title", "authors", "year"):
            if not e.get(field):
                problems.append(f"{i}: missing {field}")
        year = e.get("year")
        if year and not (isinstance(year, int) and 1990 <= year <= 2100):
            problems.append(f"{i}: implausible year {year}")
        key = (e.get("title") or "").strip().lower()
        if key in seen:
            problems.append(f"{i}: duplicate {key}")
        seen.add(key)
    return problems

refs = [{"title": "Attention Is All You Need", "authors": "Vaswani et
al.", "year": 2017},
        {"title": "attention is all you need", "authors": "Vaswani et
al.", "year": 2017},
        {"title": "LoRA", "authors": "", "year": 2021}]
print(validate(refs))
assert len(validate(refs)) == 2
```

Interview Angle

Q: Pick one paper from this list and explain its practical consequence for production systems. **A:** PagedAttention: KV cache is allocated in fixed blocks rather than contiguous per-request maxima, so internal and external fragmentation shrink and concurrency per GPU rises; it also enables prefix sharing. Trade-off: block-table indirection and kernel complexity. Check: measure achieved concurrency and KV utilisation on your own traffic rather than quoting paper figures.

Q: Which papers explain the memory and throughput numbers in Module 05? **A:** PagedAttention for KV allocation, Orca for iteration-level batching, FlashAttention for attention IO, Sarathi-Serve for chunked prefill, speculative decoding for latency, DistServe for disaggregation. Check: reproduce the KV formula and a Little's-law concurrency estimate from first principles.

Q: How do you keep up with the field without drowning? **A:** One weekly time-box, a short list of primary sources and release notes for tools you run, and a written hook per item. Trade-off: breadth versus retention; drop anything you cannot link to a decision. Check: every item has a hook within 24 hours or is discarded.

19.5 Loop Anatomy and Behavioural Stories

Conceptual Deep Dive

A typical loop has six stages, though names and order vary by company. The **recruiter screen** checks fit and logistics. A **practical or take-home** tests whether you ship working code. **LLM/ML depth** probes mechanisms (KV cache, retrieval, evaluation statistics). **System design** tests framing, sizing and trade-offs (20.1). **Behavioural and leadership** tests scope, judgement and learning. The **hiring-manager conversation** tests team fit and ownership. Scoring varies by company, so ask the recruiter what each round assesses.

Build a story bank of six to eight stories mapped to LLMOps themes: an incident you led; a trade-off decided with data; a cost saving that preserved quality; a failed experiment and what changed; influencing without authority; mentoring; disagreeing with a stakeholder; raising a safety or privacy concern. Tag each story with its themes so a coverage check reveals gaps.

Use **Situation**/Task, Action, Result, Reflection. Keep the situation short, say "I" for what you did and "we" for team context, quantify the result and end with what you changed afterwards. Each story needs a number, a decision and a learning. Aim for about two to three minutes; a monologue beyond four minutes loses the listener. For Staff, show scope across teams and long-term mechanisms (standards, templates, eval gates) instead of heroics: "I wrote the postmortem template and the rollback gate", not "I stayed up all night".

Questions to ask interviewers: how mature is evaluation and who owns the golden set; what is the on-call load for the LLM stack; what is the GPU strategy (owned, reserved, API); where do ownership boundaries lie between platform and product; how is quality measured and reported. Red flags in your own delivery: blaming, vagueness, unverifiable claims and ignoring failure.

Edge cases. No production LLM experience yet: use adjacent stories (ML, SRE, data) and project work from 18.x, honestly labelled as projects. Confidentiality: anonymise company, customer and people, keep numbers relative ("p99 halved", "cost down about a third") and avoid proprietary names. Never invent a result to fit the structure; interviewers probe specifics and inconsistencies show quickly.

Cross-Connection Scenarios

- Incident stories use the blameless postmortem structure in 17.6.
- Cost-saving stories pair with 11.8 and the levers in Module 11 sections 1.3-1.4 (legacy).
- Safety-concern stories link to governance and escalation in 16.1 and 16.5.
- Project-based stories draw on 18.2-18.6.

Production Anti-Patterns

- Stories without a result: the listener cannot judge impact, and the story scores as activity rather than outcome.
- Rehearsed monologues over four minutes: you lose interaction and the interviewer cannot probe, which reads as evasiveness.
- Claiming credit for team outcomes: a single follow-up question about your specific contribution exposes the gap and costs trust.

Minimal Working Code

A story bank with a coverage checker against the theme list.

```
THEMES = {"incident", "tradeoff", "cost"
, "failure", "influence", "mentoring",
        "disagreement", "safety"}

stories = [
    {"id": "s1", "title": "Cache leak postmortem", "tags&quo
t;: {"incident", "safety"}},
    {"id": "s2", "title": "Router saved a third of spend", "
tags": {"cost", "tradeoff"}},
    {"id": "s3", "title": "Judge disagreement with PM", "tag
s": {"disagreement"}},
]

def coverage(stories, themes=THEMES):
    covered = set().union(*(s["tags"] for s in stories))
    return sorted(themes - covered), sorted(covered - themes)

missing, unknown = coverage(stories)
print("missing themes:", missing)
assert missing == ["failure", "influence", "mentoring"] and unknown == []
```

A STAR linter that checks segment proportions, a metric and a reflection.

```
import re

def lint(story):
    words = {k: len(v.split()) for k, v in story.items()}
    total = sum(words.values())
    issues = []
    if words["situation"] / total > 0.25:
        issues.append("situation too long (>25%)")
    if words["action"] / total < 0.35:
        issues.append("action too thin (<35%)")
    if not re.search(r"\d", story["result"]):
        issues.append("result has no metric")
    if words["reflection"] < 8:
        issues.append("reflection missing or thin")
    return issues

good = {"situation": "Prod cache served stale answers.",
        "action": "I traced the cache key, added tenant scoping, wrote a replay test, and
"
                  "proposed a rollout gate that the team adopted for every cache change.",
        "result": "Stale-hit rate fell from 3% to 0.1% within a week.",
        "reflection": "I now require tenant scoping in every cache design review."}
bad = dict(good, result="It got better.", reflection="Learned.")
print(lint(bad))
assert lint(good) == [] and len(lint(bad)) == 2
```

Interview Angle

Q: Tell me about an incident you led. What changed afterwards? **A:** Give the symptom and detection time, the decision you made under uncertainty (rollback versus forward-fix), the metric that confirmed recovery, then the mechanism you installed (a gate, an alert, a runbook). Trade-off: speed of mitigation versus evidence quality. Check: time-to-detect and time-to-mitigate fell in the next incident.

Q: Describe a technical disagreement and how you resolved it. **A:** State both positions fairly, the experiment or data that discriminated between them (for example a paired evaluation), and the outcome even if you were wrong. Trade-off: speed of deciding versus evidence. Check: the decision record and the metric movement afterwards.

Q: How do you show Staff-level scope without a Staff title? **A:** Describe work that outlived you: standards, templates, review processes adopted by other teams, with adoption numbers and mentoring outcomes. Trade-off: visible heroics versus durable mechanisms. Check: other teams using the artefact without your involvement.

19.6 Glossary and Numbers Sheet

Conceptual Deep Dive

Use this as a last-minute review, not a substitute for the modules. Terms are defined in one sentence, alphabetically, with a pointer to where the repository explains them. Perishable facts (prices, model names with sizes, benchmark scores) are deliberately excluded.

Term	Definition	Where
A2A	Agent2Agent protocol for communication between independently operated agents.	9.6
ACL	Access control list: the principals allowed to read a document, enforced at retrieval time.	Module 06 section 1.8 (legacy)
Adapter	Small trainable weights added to a frozen base model, swappable per tenant or task.	Module 04 section 1.5 (legacy)

Term	Definition	Where
ASR (attack success rate)	Fraction of attack attempts that achieve the attacker's goal.	Module 08 section 2.1 (legacy)
ASR (speech recognition)	Automatic speech recognition: audio to text.	20.6
AWQ	Activation-aware weight quantisation that scales salient channels before low-bit rounding.	Module 04 section 2.4 (legacy)
BM25	Lexical ranking function with term-frequency saturation and length normalisation.	Module 06 section 2.5 (legacy)
Bundle	Versioned release unit pinning model, prompts, index, guardrails and configuration.	13.1
Burn rate	Error-budget consumption speed relative to the rate that would exactly exhaust it.	Module 10 section 2.3 (legacy)
Canary	Release to a small traffic slice with automatic rollback conditions.	13.3
Chunked prefill	Splitting long prompts into pieces interleaved with decode steps to bound ITL.	Module 05 section 1.5 (legacy)
Continuous batching	Iteration-level scheduling where requests join and leave the batch each step.	Module 05 section 1.4 (legacy)
Cross-encoder	Model scoring a query and document jointly; used for reranking.	Module 06 section 1.6 (legacy)
Decode	Token-by-token generation phase, typically memory-bandwidth bound.	Module 01 section 1.4 (legacy)
Disaggregated serving	Running prefill and decode on separate worker pools.	Module 05 section 1.9 (legacy)
DPO	Direct preference optimisation: trains on preference pairs without a separate reward model.	15.3
ELO	Rating system updating scores from pairwise outcomes.	7.6
Embedding	Dense vector representation of text used for similarity search.	Module 06 section 2.1 (legacy)
Error budget	Allowed unreliability, equal to one minus the SLO over a window.	Module 10 section 2.3 (legacy)
Eval gate	Automated pass/fail check on evaluation metrics that blocks a release.	13.2
Expert parallelism	Placing different MoE experts on different devices.	5.7
Faithfulness	Fraction of answer claims supported by the retrieved context.	Module 07 section 2.1 (legacy)
Flywheel	Loop converting production feedback into evaluation and training data.	14.4
Golden set	Curated labelled evaluation set used for regression gating.	14.1

Term	Definition	Where
Goodput	Throughput counting only requests that meet their latency SLOs.	Module 05 section 3.9 (legacy)
GQA	Grouped-query attention: query heads share fewer KV heads, shrinking the KV cache.	Module 01 section 2.3 (legacy)
GRPO	Group relative policy optimisation: RL using group-normalised rewards instead of a value model.	15.4
Guardrail	Control that validates or constrains inputs, outputs or actions.	Module 08
HITL	Human in the loop: a person approves or reviews selected agent actions.	Module 09 section 2.7 (legacy)
HNSW	Hierarchical navigable small world graph index for approximate nearest neighbours.	Module 06 section 1.3 (legacy)
Hybrid search	Combining lexical and dense retrieval, often fused with RRF.	Module 06 section 1.5 (legacy)
Idempotency	Repeating an operation with the same key has the same effect as doing it once.	Module 09 section 2.6 (legacy)
ITL	Inter-token latency: the gap between consecutive output tokens.	Module 10 section 1.5 (legacy)
Judge	An LLM used as an evaluator of other model outputs.	Module 07 section 1.5 (legacy)
Kill switch	Operational control that disables a feature or tool immediately.	13.6
KV cache	Stored attention keys and values for past tokens, reused during decode.	Module 01 section 2.3 (legacy)
LoRA	Low-rank adaptation: trains small low-rank update matrices on a frozen base.	Module 04 section 2.6 (legacy)
MCP	Model Context Protocol: standard for exposing tools, resources and prompts to LLM hosts.	Module 09 section 1.7 (legacy)
MFU	Model FLOPs utilisation: achieved model FLOPs divided by hardware peak.	15.2
MoE	Mixture of experts: each token is routed to a subset of expert feed-forward blocks.	5.7
MRR	Mean reciprocal rank of the first relevant result.	Module 06 section 2.6 (legacy)
MTBF / MTTR	Mean time between failures / mean time to recover.	17.5
NF4	4-bit NormalFloat data type used by QLoRA.	Module 04 section 3.5 (legacy)
p95 / p99	The 95th and 99th percentile of a distribution, typically of latency.	Module 10 section 2.2 (legacy)
PagedAttention	KV cache stored in fixed-size blocks addressed through block tables.	Module 05 section 1.3 (legacy)

Term	Definition	Where
Pipeline parallelism	Splitting model layers across devices in stages.	Module 05 section 1.8 (legacy)
Prefill	Compute-bound processing of the prompt before the first output token.	Module 01 section 1.4 (legacy)
Prefix cache	Reuse of KV blocks for a shared prompt prefix.	5.6
Prompt injection	Untrusted text that tries to override the system's instructions.	Module 08 section 1.2 (legacy)
PSI	Population stability index: drift statistic on binned distributions.	Module 10 section 2.5 (legacy)
QLoRA	LoRA training on a 4-bit quantised frozen base model.	Module 04 section 3.6 (legacy)
RAG	Retrieval-augmented generation: grounding generation in retrieved documents.	Module 06
RAGAS	Open-source framework of reference-free RAG metrics.	Module 07 section 3.3 (legacy)
Recall@k	Fraction of relevant items found in the top k results.	Module 06 section 2.2 (legacy)
Reranker	Second-stage model that reorders candidates for precision.	Module 06 section 1.6 (legacy)
RLHF	Reinforcement learning from human feedback via a learned reward model.	15.3
RPS / QPS	Requests / queries per second; treat as synonyms unless a system defines a query differently.	Module 01 section 3.2 (legacy)
RRF	Reciprocal rank fusion: scores items by the sum of 1/(k + rank) across lists.	Module 06 section 2.4 (legacy)
Semantic cache	Cache keyed by embedding similarity rather than exact text.	Module 11 section 1.3 (legacy)
SFT	Supervised fine-tuning on instruction-response pairs.	15.1
Shadow deployment	New version receives copied traffic whose outputs are not served.	13.3
SLI	Service-level indicator: a measured quantity such as the fraction of fast requests.	Module 10 section 2.3 (legacy)
SLO	Service-level objective: the target for an SLI over a window.	Module 10 section 2.3 (legacy)
Speculative decoding	A draft model proposes tokens that the target model verifies in one pass.	Module 05 section 1.6 (legacy)
SRM	Sample ratio mismatch: observed traffic split differs from the planned split, invalidating an experiment.	13.5
Tensor parallelism	Splitting each layer's matrices across devices within a node.	Module 05 section 1.8 (legacy)
TPM	Tokens per minute, commonly a provider rate-limit unit.	11.7

Term	Definition	Where
TPOT	Time per output token: (end-to-end time minus TTFT) divided by (output tokens minus one), an average.	Module 10 section 1.5 (legacy)
TPS	Tokens per second, either per request or system-wide; state which.	Module 05 section 3.9 (legacy)
TTFT	Time to first token: from request arrival to the first output token.	Module 10 section 1.5 (legacy)
Vector database	Store optimised for approximate nearest-neighbour search with metadata filtering.	Module 06 section 1.2 (legacy)
W4A16	4-bit weights with 16-bit activations.	Module 04 section 1.2 (legacy)
WER	Word error rate: word-level edit distance over reference length.	20.6
Wilson interval	Confidence interval for a proportion that behaves well at small n and extreme rates.	Module 10 section 2.7 (legacy)

Numbers sheet (extends Module 12 section 1.6 (legacy); rules of thumb are flagged). Little's law: concurrency L = arrival rate x time in system. Wilson 95% interval for a pass rate p over n: centre (p + z^2/2n)/(1 + z^2/n), half-width z sqrt(p(1-p)/n + z^2/4n^2)/(1 + z^2/n). Burn rate = observed error rate / (1 - SLO). Young/Daly checkpoint interval, to first order: sqrt(2 x checkpoint cost x MTBF). Speculative speedup: expected tokens per verification (1 - a^(g+1))/(1 - a) for acceptance a and g draft tokens, divided by relative cost g c + 1. KV bytes per token = 2 x layers x KV heads x head dim x bytes. ZeRO per-GPU bytes for Adam mixed precision over N GPUs and P parameters: stage 1 about 4P + 12P/N, stage 2 about 2P + 14P/N, stage 3 about 16P/N. Unit pitfalls: bits versus bytes, GB versus GiB, input versus output tokens, tokens per second per request versus system-wide.

Look-alikes	Disambiguation
TPS vs TPM	Per second vs per minute; TPM is usually a quota, TPS a speed
ITL vs TPOT	ITL: individual gaps; TPOT: an average over a response
RPS vs QPS	Usually equivalent; check whether a "query" fans out to many calls
ASR	Attack success rate or speech recognition

Vendors rename the same ideas (prompt caching, context caching, prefix caching); describe the mechanism, then name the vendor feature. In RL the same letters (PPO, GRPO) denote algorithms, but "reward model" in RLHF is a learned scorer, whereas RLVR uses programmatic verifiers (15.4).

Cross-Connection Scenarios

- Formulas here are used in the estimation drills in 12.6 and the worked sizing in 20.2.
- Latency terms underpin voice budgets in 20.6 and serving benchmarks in 5.8.
- Governance terms expand in 16.1-16.3.
- Use the glossary entries as flashcards in the Leitner scheduler in 19.2.

Production Anti-Patterns

- Memorising terms without being able to use them in a sentence: definitions collapse when the interviewer asks "so what happens when it fails?".
- Mixing up TTFT and TPOT definitions: capacity and SLO answers become wrong by an order of magnitude on long outputs.
- Copying numbers without units: bits versus bytes or GB versus GiB errors silently cost 8x or about 7%.

Minimal Working Code

A glossary linter plus search.

```
def lint(entries, max_len=140):
    problems, terms = [], [t for t, _ in entries]
    if terms != sorted(terms, key=str.lower):
        problems.append("not alphabetical")
    dupes = {t for t in terms if terms.count(t) > 1}
    problems += [f"duplicate: {t}" for t in sorted(dupes)]
    problems += [f"too long: {t}" for t, d in entries if len(d) > max_len]
    return problems

def search(entries, query):
    q = query.lower()
    return [t for t, d in entries if q in t.lower() or q in d.lower()]

G = [("ITL", "Gap between consecutive output tokens."),
     ("TPOT", "Average time per output token over a response."),
     ("TTFT", "Time from request arrival to the first output token.")]
print(lint(G), search(G, "output"))
assert lint(G) == [] and search(G, "first") == ["TTFT"]
assert lint(G[::-1]) == ["not alphabetical"]
```

Quick-calculator functions that reproduce the Module 12 cheatsheet numbers.

```
import math

def kv_bytes_per_token(layers, kv_heads, head_dim, bytes_per=2):
    return 2 * layers * kv_heads * head_dim * bytes_per

def weight_bytes(params, bytes_per):
    return params * bytes_per

def littles_law(rate, seconds):
    return rate * seconds

def wilson(p, n, z=1.96):
    denom = 1 + z * z / n
    centre = (p + z * z / (2 * n)) / denom
    half = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / denom
    return centre - half, centre + half

def burn_rate(error_rate, slo):
    return error_rate / (1 - slo)

assert kv_bytes_per_token(32, 8, 128) == 128 * 1024          # 8B-class, GQA, BF16
assert kv_bytes_per_token(80, 8, 128) == 320 * 1024          # 70B-class
assert weight_bytes(8e9, 2) == 16e9
assert littles_law(20, 5) == 100
lo, hi = wilson(0.8, 100)
assert round(lo, 2) == 0.71 and round(hi, 2) == 0.87
assert round(burn_rate(0.0144, 0.999), 1) == 14.4
print("ok")
```

Interview Angle

Q: Define goodput and explain why it beats throughput as an SLO-aware metric. **A:** Goodput counts only completed work that met its latency targets (for example requests with TTFT and ITL inside SLO). Raw throughput can rise while users suffer, for instance by over-batching. Trade-off: goodput depends on the chosen SLO and needs per-request timing. Check: plot goodput versus offered load; its peak, not the throughput peak, sets capacity.

Q: Explain TTFT, ITL and TPOT precisely. **A:** TTFT is arrival to first output token and includes queueing and prefill. ITL is the gap between two consecutive output tokens, so it has a distribution (use p95/p99). TPOT is (end-to-end minus TTFT)/(output tokens minus one), an average that hides stalls. Check: compute all three from the same timestamps.

Q: Compute KV cache per token for a model from its config. **A:** 2 (K and V) x layers x KV heads x head dimension x bytes per element; for 32 layers, 8 KV heads, head dimension 128 in BF16 that is 131,072 bytes, 128 KiB. Multiply by context and concurrency to size memory; GQA and KV quantisation (4.6) cut it. Check: compare with measured free memory after weights.